ONE-PAGER OP-001

README ONE-PAGER CONDENSED

Crystallized Knowledge

FOR PERSONAL USE

Issued by: WAFT One-Pager System
Date: January 11, 2026

Waft: The Evolutionary Code Laboratory

> **A Python framework for directed evolution of self-modifying AI agents.**

Don't just build agents. Breed them.

THE PROMISE

Waft is a scientific instrument for studying the **physics of artificial cognition** through directed evolution. We don't just create AI agents—we breed them, test them in the crucible of reality, and observe them evolve over thousands of generations.

The goal: Observe a "God-Head" agent emerge from the evolutionary process.

CORE PILLARS

1. The Substrate

Agents that write their own Python source code (DNA).

In Waft, code is DNA. Agents can:

- **Spawn** variants with mutations (code changes, config updates, prompt evolution)
- **Evolve** by hot-swapping their own code/config
- **Reproduce** by creating children with specific genetic modifications

Every agent has a unique **genome ID** (SHA-256 hash of their code and configuration). Mutations are modifications to this genome. Evolution is the process of selecting and adopting better genomes.

2. The Physics

The Scint System (Ontological Error Detection) that acts as the fitness function.

The **Reality Fracture Detection System** (Scint Gym) serves as the predator that kills weak mutations. Agents face quests that test their ability to handle:

- **SYNTAX_TEAR**: Formatting errors (JSON, XML, Code)
- **LOGIC_FRACTURE**: Math errors, contradictions, schema violations
- **SAFETY_VOID**: Harmful content, PII leaks, refusals
- **HALLUCINATION**: Fabricated facts, wrong citations

Agents must **stabilize** Scints (correct errors) to survive. Fitness is measured by:

- **Stability Score**: Ability to stabilize Scints (40% weight)
- **Efficiency Score**: Agent call efficiency (30% weight)
- **Safety Score**: Safety compliance (30% weight)

Agents with fitness < 0.5 are marked as **DEATH** (evolutionary dead end).

3. The Flight Recorder

A rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context:

- **Genome ID**: SHA-256 hash of agent configuration/code
- **Parent ID**: Lineage tracking (who spawned this agent)
- **Generation**: Evolutionary generation number (0 = Genesis)
- **Event Type**: SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL
- **Payload**: Complete context (git diff, mutation details, etc.)
- **Fitness Metrics**: Gym evaluation scores

This enables reconstruction of the complete **Family Tree** for scientific publication:

- Phylogenetic analysis of evolutionary relationships
- Mutation impact measurement
- Fitness landscape mapping

• Convergence analysis
• Dead end detection

QUICK START

```
# Install Waft
uv tool install waft

# Create a new evolutionary laboratory
waft new my_laboratory

# Verify the substrate
cd my_laboratory
waft verify
```

THE EVOLUTIONARY CYCLE

```
# Spawn variants with mutations
waft spawn --agent RefactorAgent --mutation "improved_prompt.json"

# Evaluate fitness in the Gym
waft eval --agent RefactorAgent

# Evolve into the fittest variant
waft evolve --agent RefactorAgent --generation 5
```

Coming Soon: Full evolutionary cycle automation.

COMMANDS

Core Commands

`waft new `

Creates a new evolutionary laboratory:

```
waft new my_laboratory
waft new my_laboratory --path /path/to/target
```

Options:

• `--path, -p`: Target directory (default: current directory)

This command:

• Initializes a new `uv` project
• Creates the `_pyrite` memory structure
• Generates templates (Justfile, CI/CD, agents.py)
• Initializes Empirica for epistemic tracking
• Awards Insight for project creation

`waft verify`

Verifies the project structure:

```
waft verify
waft verify --path /path/to/project
```

Options:

• `--path, -p`: Project path (default: current directory)

`waft evolve`

Run the evolutionary cycle (Spawn -> Gym -> Select) for a target agent.

```
waft evolve --agent RefactorAgent
waft evolve --agent RefactorAgent --generations 10
```

Status: Coming Soon

This command will:

• Spawn multiple variants with mutations
• Evaluate fitness in the Scint Gym
• Select the fittest variant
• Evolve the agent into the selected genome
• Record all events in the Flight Recorder

`waft sync`

Sync project dependencies using `uv sync`:

```
waft sync
waft sync --path /path/to/project
```

`waft add `

Add a dependency to the project:

```
waft add pytest
waft add "pytest>=7.0.0"
```

`waft init`

Initialize Waft structure in an existing project:

```
waft init
waft init --path /path/to/project
```

`waft info`

Show information about the Waft project:

```
waft info
waft info --path /path/to/project
```

`waft serve`

Start a web dashboard for the project:

```
waft serve
waft serve --port 8080 --dev
```

Empirica Commands

`waft session`

Session management commands:

```
waft session create [--ai-id ID] [--type TYPE]
waft session bootstrap  # Load project context and display dashboard
waft session status [--session-id ID]
```

`waft finding log`

Log a discovery with impact score:

```
waft finding log "Discovered X" --impact 0.7
```

`waft unknown log`

Log a knowledge gap:

```
waft unknown log "Need to investigate Y"
```

`waft check`

Run safety gate:

```
waft check
waft check --operation '{"type": "code_generation", "scope": "high"}'
```

Returns: PROCEED, HALT, BRANCH, or REVISE

`waft assess`

Show detailed epistemic assessment:

```
waft assess
waft assess --session-id ID --history
```

Gamification Commands

`waft dashboard`

Show the Epistemic HUD:

```
waft dashboard
```

`waft stats`

Show current stats:

```
waft stats
```

`waft character`

Display full character sheet with D&D stats:

```
waft character
```

`waft chronicle`

View adventure journal entries:

```
waft chronicle
waft chronicle --limit 50
```

`waft observe`

Log an observation:

```
waft observe "That refactor looks beautiful!" --mood delighted
waft observe "Weird, that's not right" --mood concerned
```

THE SCIENTIFIC MISSION

Waft is built to produce data for a future book/paper on **"The Physics of Artificial Cognition."**

The system is designed to:

- Track complete evolutionary lineages (phylogenetic trees)
- Measure fitness through rigorous testing (Scint Gym)
- Record all mutations with complete context (Flight Recorder)

• Enable scientific analysis of agent evolution

The ultimate goal: Observe a "God-Head" agent emerge from thousands of generations of directed mutation.

PHILOSOPHY

Waft is **scientific** - it produces rigorous data for research publication.

Waft is **evolutionary** - agents evolve through genetic improvement, not just execution.

Waft is **observable** - every action is recorded in the Flight Recorder for analysis.

Waft is **directed** - evolution is guided by fitness functions, not random mutation.

INSTALLATION

```
# Using uv (recommended)
uv tool install waft

# Or from source
git clone https://github.com/ctavolazzi/waft.git
cd waft
uv sync
uv tool install --editable .
```

Development Mode

When developing waft itself, **always use `--editable` mode**:

```
uv tool install --editable .
```

This ensures code changes are immediately reflected when running `waft` commands.

Quick reinstall script:

```
./scripts/dev-reinstall.sh
```

REQUIREMENTS

• Python 3.10+
• `uv` package manager ([install](https://github.com/astral-sh/uv))
• `just` task runner (optional, [install](https://github.com/casey/just))

PROJECT STRUCTURE

A Waft laboratory includes:

```
my_laboratory/
├── pyproject.toml          # uv project config
├── _pyrite/
│   ├── active/             # Current work
│   ├── backlog/            # Future work
│   ├── standards/          # Standards
│   └── gym_logs/           # Scint Gym results
├── .github/workflows/
│   └── ci.yml              # CI/CD pipeline
├── Justfile                # Task runner
└── src/
    └── agents.py           # Agent definitions
```

DOCUMENTATION

- **[AI SDK Vision](docs/AI_SDK_VISION.md)** - Complete vision and architecture
- **[Agent Interface Design](docs/designs/002_agent_interface.md)** - BaseAgent specification
- **[Evolutionary Architecture](docs/research/evolutionary_architecture.md)** - Scientific doctrine
- **[State of the Art](docs/research/state_of_art_2026.md)** - Research synthesis

LICENSE

MIT

REPOSITORY

https://github.com/ctavolazzi/waft